

Meeting Summary: Jakarta EE Future Directions — Jakarta REST 5.0 CDI Integration

Date: April 30, 2026

Topic: Jakarta REST 5.0 changes targeted for Jakarta EE 12

Presenter: James Perkins, IBM

Participants heard in discussion: Neil Patterson, James Perkins, Michael Redlich, Jared Anderson, Steve Butler

Purpose

James Perkins presented proposed Jakarta REST 5.0 changes focused on **full CDI integration**, with the goal of simplifying dependency injection across Jakarta EE by moving away from Jakarta REST-specific injection mechanisms such as `@Context` and toward standard CDI `@Inject`.

Key Proposed Changes

The main proposal is to make Jakarta REST more consistently CDI-based:

Area	Proposed Direction
<code>@Context</code>	Deprecated in favor of <code>@Inject</code>
Field injection	Replace <code>@Context</code> with <code>@Inject</code>
Constructor injection	Add <code>@Inject</code> to constructors and remove <code>@Context</code> from parameters
Method parameters	Remove <code>@Context</code> ; avoid putting <code>@Inject</code> on resource methods
<code>@Suspended</code>	Deprecated, with CDI type inference used instead
Context resolvers	Deprecated in favor of CDI producers
<code>@Path</code> , <code>@Provider</code> , <code>@ApplicationPath</code>	Become CDI stereotypes / bean-defining annotations
Default scopes	Resources default to request scope; providers and applications default to application scope
Target release	Jakarta EE 12 / Jakarta REST 5.0

Rationale

Neil asked what benefit organizations would get from these changes. James explained that the primary benefit is **simplification and consistency**: developers would no longer need to know when to use Jakarta REST-specific `@Context` versus CDI `@Inject`. The long-term goal is to align Jakarta REST with the broader Jakarta EE movement toward CDI as the common injection model.

Neil summarized the benefit as making injection clearer and simpler for developers because they would have one consistent approach across specs.

Compatibility and Breaking Change Concerns

A substantial part of the discussion focused on backward compatibility.

Jared noted that if annotations such as `@HeaderParam`, `@QueryParam`, and related parameter annotations become CDI qualifiers, then requiring `@Inject` on fields or constructors could become a breaking change. James agreed and said implementations could potentially support both the old and new approaches, but the spec and TCK would need to be aligned.

Key compatibility concern:

If the TCK continues testing the old style, implementations must continue supporting it. If the tests are removed, implementations may not be required to preserve backward compatibility.

James said he would clarify this in the pull request.

Resource Method Injection and `@EntityParam`

James introduced a possible new `@EntityParam` annotation to distinguish the request body from CDI beans in resource method parameters.

Example issue discussed:

```
public Response createOrder(Customer customer, CustomerValidator validator)
```

Without a distinguishing annotation, the runtime needs to know whether `Customer` is the HTTP request body or a CDI bean. `@EntityParam` would mark the request-body parameter explicitly.

However, this raised a compatibility concern because existing Jakarta REST methods already rely on implicit entity-body detection. Requiring `@EntityParam` would likely break existing applications.

Emerging Direction: Defer `@EntityParam`

Jared suggested a staged approach:

1. In Jakarta REST 5.0, do **not** allow arbitrary CDI bean injection into resource method parameters.
2. Preserve current entity-parameter behavior and avoid introducing `@EntityParam` as a required migration step.
3. Revisit full method-parameter CDI bean injection in Jakarta REST 6.0, when deprecated APIs such as `@Context` may be removed.

James agreed this approach made sense and said he was leaning toward backing away from `@EntityParam` for 5.0 to preserve compatibility.

Scope Discussion

Michael asked whether request scope had always been the default for resources or whether that was new in 5.0.

James said it is effectively new as an explicit specification requirement. Resource classes have generally behaved as request-scoped in practice, but this was not clearly stated in the specification. Providers and applications typically behave as application-scoped.

Jared also asked about application-scoped resources. James clarified:

- Application-scoped resources can work.
- Method parameter injection should be safe because methods are invoked per request.
- Constructor or field injection of request-specific values such as query parameters or headers can produce unexpected results in application-scoped resources.

`Application#getClasses()` and `getSingletons()`

James discussed how CDI-managed resources interact with `Application`.

Key points:

- `getSingletons()` is already deprecated.
- Objects returned from `getSingletons()` are created by the application implementation and will not be CDI beans.
- `getClasses()` will control REST routing, but classes annotated with `@Path` or `@Provider` may still be CDI beans.
- If `getClasses()` is not specified, all discovered CDI-managed REST resources are routable.

Migration Guidance Discussed

James outlined a simple migration model:

Existing Pattern	Migration Direction
<code>@Context</code> on fields	Replace with <code>@Inject</code>
<code>@Context</code> on constructors	Remove <code>@Context</code> ; add <code>@Inject</code> to constructor
<code>@Context</code> on method parameters	Remove <code>@Context</code>
<code>@Suspended</code>	Remove annotation

Existing Pattern	Migration Direction
Request body parameters	Keep current behavior for now; likely avoid requiring <code>@EntityParam</code> in 5.0
CDI scopes	Explicit scopes may be used, but developers should be cautious with application-scoped resources

Tools and Follow-up Resources

James mentioned that he had created an **OpenRewrite recipe** to help convert the TCK and potentially user code. He said it may not belong directly in the Jakarta REST spec source but could be contributed to OpenRewrite or otherwise published publicly.

He also offered to share slides and links to the pull request.

Decisions / Outcomes

The meeting did not finalize the specification, but the strongest conclusions were:

1. **Proceed with CDI integration for Jakarta REST 5.0.**
2. **Deprecate `@Context` and move toward `@Inject`.**
3. **Preserve as much backward compatibility as possible.**
4. **Reconsider or defer `@EntityParam` for Jakarta REST 5.0** because it could introduce breaking changes.
5. **Potentially target fuller CDI method-parameter injection for Jakarta REST 6.0.**
6. **Clarify TCK expectations** so compatibility requirements are explicit.
7. **James will research CDI qualifier requirements**, especially whether annotations such as `@HeaderParam` require `@Inject` in all relevant cases.

Action Items

Owner	Action
James Perkins	Clarify backward-compatibility expectations in the pull request.
James Perkins	Reconsider introducing <code>@EntityParam</code> in Jakarta REST 5.0; likely defer to 6.0.
James Perkins	Research CDI qualifier behavior and whether <code>@Inject</code> is required with parameter annotations such as <code>@HeaderParam</code> .
James Perkins	Share slides and pull request links with the group.
James Perkins	Determine where the OpenRewrite migration recipe should live or be published.
Neil Patterson / Michael Redlich	Coordinate the next Future Directions session.